

SIMATS ENGINEERING

ASSESSMENT TOOL – III : CODING CHALLENGE

Course Code	ECA14
Course Name	Embedded Systems
Course Outcome	CO2
Assessment Tool	Coding Challenge
Question 1 No.	17
Question 1 Title	Design an Embedded C application for EEPROM-based data logging and evaluate memory management strategies.
Question 2 No.	18
Question 2 Title	Construct a modular Embedded C program using multiple files for a multi-sensor system and analyze scalability.
Maximum Marks	100 (50 + 50)
Bloom's Taxonomy Level	BL3 / BL4

Evaluation Rubric – Question 1 (50 Marks)

Criteria	Weightage	Level 4	Level 3	Level 2	Level 1	Marks Awarded
Problem Understanding	10	☐ 10	☐ 8	☐ 6	☐ 4	
Algorithm	12.5	☐ 12.5	☐ 10	☐ 8	☐ 5	
Code Implementation	10	☐ 10	☐ 8	☐ 6	☐ 4	
Competitive Performance	10	☐ 10	☐ 8	☐ 6	☐ 4	
Output Validation	7.5	☐ 7.5	☐ 6	☐ 4	☐ 2	
Total – Question 1	50					____ /50

Evaluation Rubric – Question 2 (50 Marks)

Criteria	Weightage	Level 4	Level 3	Level 2	Level 1	Marks Awarded
Problem Understanding	10	☐ 10	☐ 8	☐ 6	☐ 4	
Algorithm	12.5	☐ 12.5	☐ 10	☐ 8	☐ 5	
Code Implementation	10	☐ 10	☐ 8	☐ 6	☐ 4	
Competitive Performance	10	☐ 10	☐ 8	☐ 6	☐ 4	
Output Validation	7.5	☐ 7.5	☐ 6	☐ 4	☐ 2	
Total – Question 2	50					____ /50

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Embedded Systems Laboratory Report

Question 17

Design an Embedded C application for EEPROM-based data logging and evaluate memory management strategies.

Aim

To design and simulate an ESP32-based data-logging application that stores sensor readings in EEPROM and to evaluate efficient memory-management strategies.

Objectives

- Acquire an analog sensor value using the ESP32 ADC.
- Store each reading as a structured EEPROM record with sequence number and time stamp.
- Read and display all stored records through the Serial Monitor.
- Clear stored records through a dedicated control input.
- Apply fixed-size records, metadata, bounds checking, and circular-buffer management.

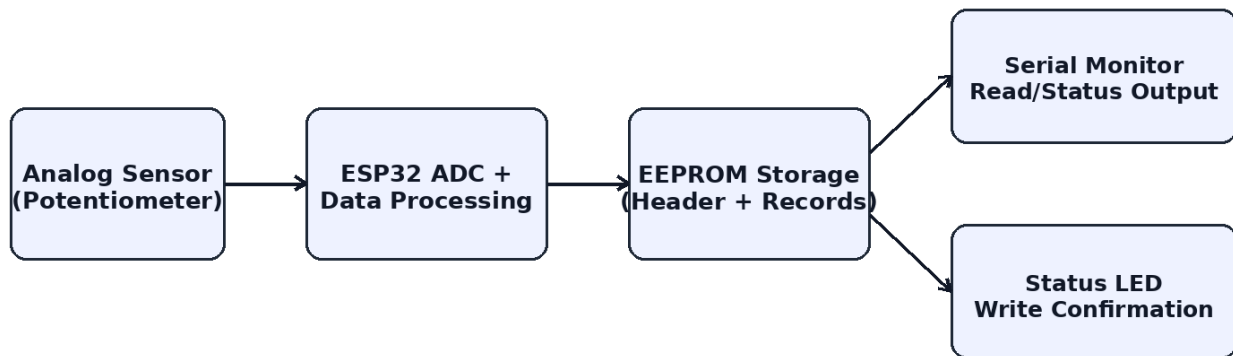
Requirements

Hardware: ESP32 DevKit, potentiometer as analog sensor, three push buttons, one status LED, 220 ohm resistor and connecting wires.

Software: Wokwi Simulator, Arduino C++ / Embedded C, EEPROM library and Serial Monitor.

Theory

EEPROM-Based Data Logging - Block Diagram



LOG stores data | READ displays records | CLEAR erases memory

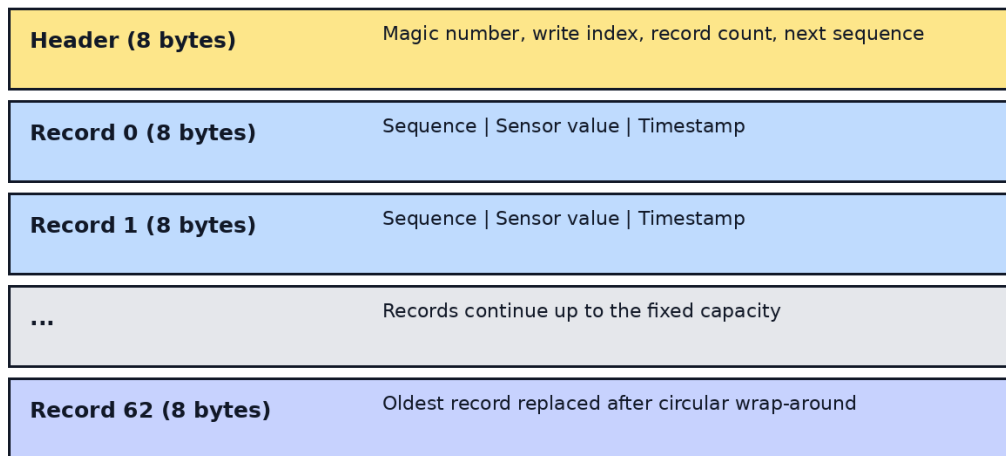
Figure 1: Block diagram of the EEPROM-based data-logging system.

EEPROM provides non-volatile storage, so recorded values remain available after reset. On ESP32, the EEPROM library emulates EEPROM using flash memory. To manage this storage safely, the program reserves a header region for metadata and divides the remaining space into equal-size records.

A circular buffer prevents memory overflow. When the maximum number of records is reached, the write index returns to the first record and the oldest value is replaced. This keeps memory usage predictable and avoids invalid addresses.

Memory Layout

EEPROM Memory Organization Diagram



Circular buffer keeps the memory bounded and prevents address overflow.

Figure 2: EEPROM header and circular fixed-record memory organization.

Memory Region	Purpose	Size
Header	Magic number, write index, record count, next sequence	8 bytes
Record Area	Sequence, sensor value and time stamp	8 bytes/record
Total EEPROM	Configured emulated EEPROM capacity	512 bytes
Maximum Records	$(512 - 8) / 8$	63 records

Algorithm

EEPROM Data Logger Flowchart

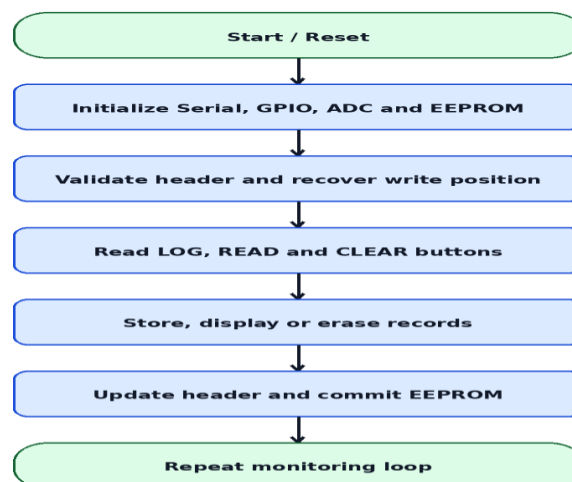


Figure 3: Flowchart of the EEPROM logging, reading and clearing process.

- Initialize Serial communication, GPIO pins, ADC and EEPROM.
- Read the EEPROM header and validate the magic number.
- If the memory is uninitialized, create a new header.
- On LOG button press, read the sensor and create a new record.
- Write the record at the current write index and update the header.
- Wrap the index to zero when the end of the record area is reached.
- On READ button press, display records from oldest to newest.
- On CLEAR button press, erase EEPROM and reinitialize the header.

Testing and Validation

Test Case	Input/Action	Expected Result
1	Start system	EEPROM capacity and initialization status displayed
2	Adjust sensor and press LOG	New record stored and LED flashes
3	Press READ	All saved records displayed in sequence
4	Press CLEAR	Records deleted and header reset
5	Store more than 63 records	Oldest records overwritten safely

Memory Management Evaluation

- Fixed-size records provide deterministic address calculation and eliminate fragmentation.
- The circular buffer prevents overflow and supports continuous logging.
- A metadata header enables recovery of write position and record count after reset.
- Controlled writes reduce flash wear because data is committed only when requested.
- Bounds checking prevents invalid record addresses and protects stored data.
- A single commit after data and metadata updates keeps the stored state consistent.
- For larger products, wear levelling, checksums, double-buffered headers or external FRAM can improve reliability.

Result

The EEPROM data logger successfully stored, displayed and cleared sensor readings. The memory remained bounded, and the circular-buffer strategy ensured continuous logging without address overflow.

Conclusion

The experiment demonstrates non-volatile data storage and practical memory management in an embedded application. The selected design is simple, predictable and suitable for moderate logging rates. The EEPROM-based data logger successfully stores sensor readings even after system reset or power interruption. The use of fixed-size records provides predictable memory allocation and avoids fragmentation. The circular-buffer method prevents memory overflow by replacing the oldest data when the storage becomes full. Header information such as record count and write position helps the system recover stored data correctly after restarting. Controlled EEPROM writes and commit operations also reduce unnecessary flash usage and improve reliability. Overall, the selected design is simple, efficient, scalable, and suitable for moderate data-logging applications such as environmental monitoring, industrial sensing, and automotive systems.

Program Code - EEPROM Data Logger

```
#include <EEPROM.h>
#define SENSOR 34
#define LOG_BTN 18
#define READ_BTN 19
#define CLEAR_BTN 21
#define LED_PIN 25
#define EEPROM_SIZE 512
#define MAGIC 0xA55A

struct Header {
  uint16_t magic, writeIndex, count, nextSeq;
};
struct Record {
  uint16_t sequence, value;
  uint32_t timeStamp;
};

Header header;
const int START = sizeof(Header);
const int MAX_RECORDS =
  (EEPROM_SIZE - START) / sizeof(Record);

void saveHeader() {
  EEPROM.put(0, header);
  EEPROM.commit();
}

void initializeMemory() {
  EEPROM.get(0, header);
  if (header.magic != MAGIC ||
    header.writeIndex >= MAX_RECORDS ||
    header.count > MAX_RECORDS) {
    header = {MAGIC, 0, 0, 1};
    saveHeader();
  }
}

void logData() {
  Record r;
  r.sequence = header.nextSeq++;
  r.value = map(analogRead(SENSOR), 0, 4095, 0, 100);
  r.timeStamp = millis();

  int address = START +
    header.writeIndex * sizeof(Record);
  EEPROM.put(address, r);

  header.writeIndex =
    (header.writeIndex + 1) % MAX_RECORDS;
  if (header.count < MAX_RECORDS) header.count++;
  saveHeader();

  digitalWrite(LED_PIN, HIGH);
  delay(150);
  digitalWrite(LED_PIN, LOW);

  Serial.print("Stored Record ");
  Serial.print(r.sequence);
  Serial.print(" | Value: ");
  Serial.print(r.value);
  Serial.println("");
}

void readData() {
  if (header.count == 0) {
    Serial.println("No records available");
    return;
  }

  int oldest =
    (header.count == MAX_RECORDS) ?
    header.writeIndex : 0;

  for (int i = 0; i < header.count; i++) {
    int index = (oldest + i) % MAX_RECORDS;
    Record r;
    EEPROM.get(START + index * sizeof(Record), r);

    Serial.print("Record ");
    Serial.print(r.sequence);
    Serial.print(" | Value: ");
    Serial.print(r.value);
    Serial.print(" | Time: ");
    Serial.println(r.timeStamp);
  }
}

void clearData() {
  header = {MAGIC, 0, 0, 1};
  saveHeader();
  Serial.println("EEPROM records cleared");
}

void setup() {
  Serial.begin(115200);
  pinMode(LOG_BTN, INPUT_PULLUP);
  pinMode(READ_BTN, INPUT_PULLUP);
  pinMode(CLEAR_BTN, INPUT_PULLUP);
  pinMode(LED_PIN, OUTPUT);

  analogReadResolution(12);
  EEPROM.begin(EEPROM_SIZE);
  initializeMemory();

  Serial.println("EEPROM DATA LOGGING SYSTEM");
  Serial.print("Maximum Records: ");
  Serial.println(MAX_RECORDS);
}

void loop() {
  if (!digitalRead(LOG_BTN)) {
    logData();
    delay(300);
  }
  if (!digitalRead(READ_BTN)) {
    readData();
    delay(300);
  }
}
```

```
if (!digitalRead(CLEAR_BTN)) {
  clearData();
  delay(300);
}
```

Simulation Output

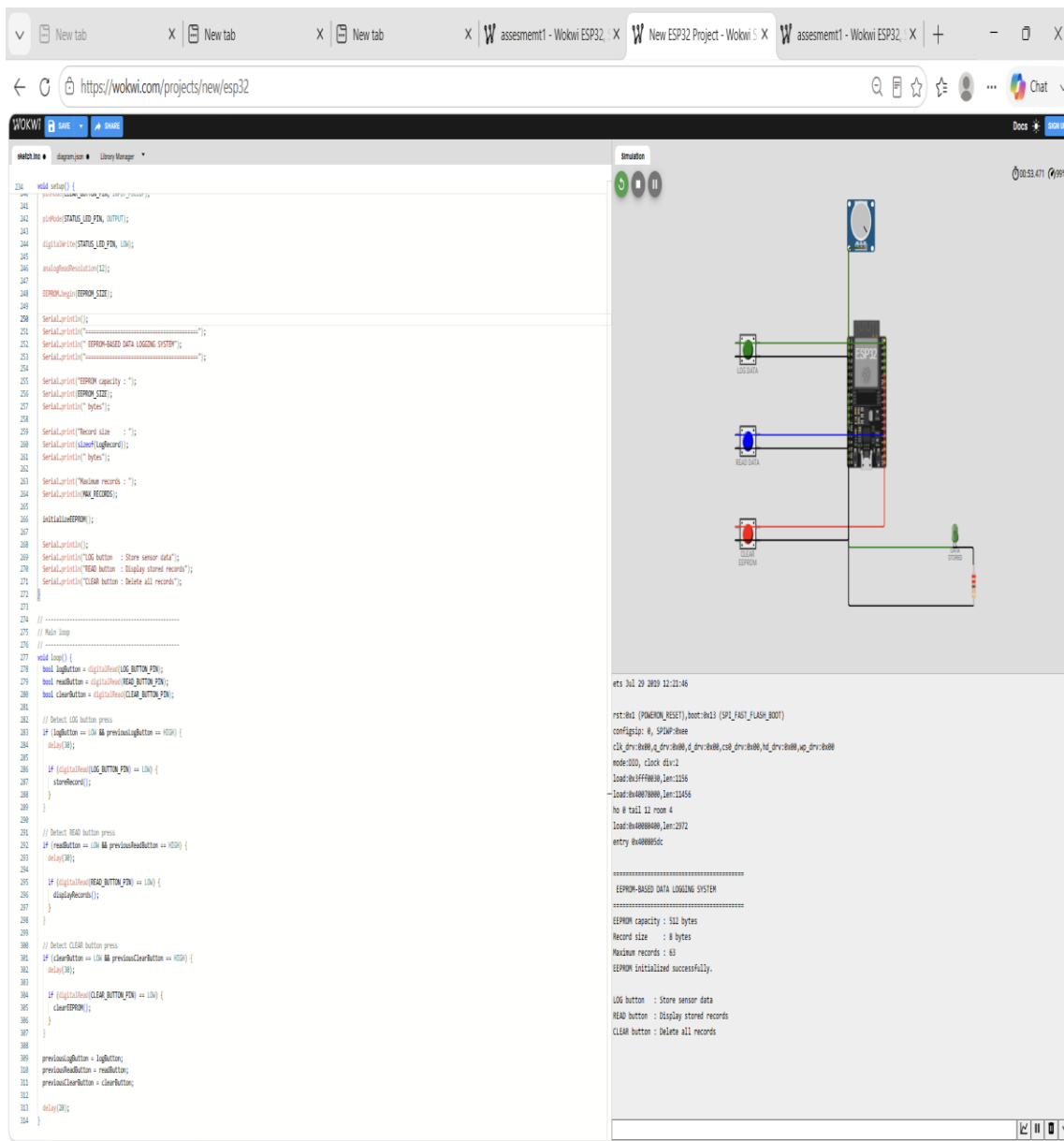


Figure 1: EEPROM-based data-logging circuit and Serial Monitor output in Wokwi.

Embedded Systems Laboratory Report

Question 18

Construct a modular Embedded C program using multiple files for a multi-sensor system and analyze scalability.

Aim

To develop a modular multi-sensor monitoring application for ESP32 and to analyze how modular design improves maintainability and scalability.

Objectives

- Monitor temperature, gas and light values using three analog inputs.
- Provide separate warning indications through LEDs and a buzzer.
- Organize functionality into sensor, alert and application modules.
- Demonstrate how new sensors and actuators can be added with minimum code changes.
- Evaluate code reuse, maintenance, memory use and system expansion.

Requirements

Hardware: ESP32 DevKit, three potentiometers representing sensors, three LEDs, three 220 ohm resistors, one buzzer and connecting wires.

Software: Wokwi Simulator and Arduino C++ / Embedded C.

Modular Architecture

Multiple-File Software Architecture

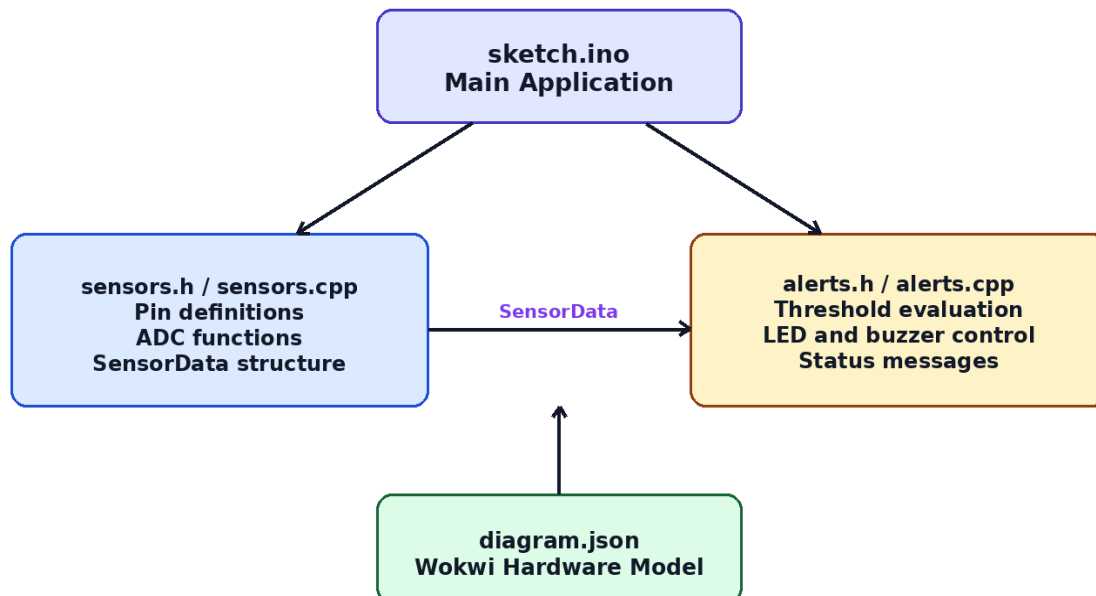
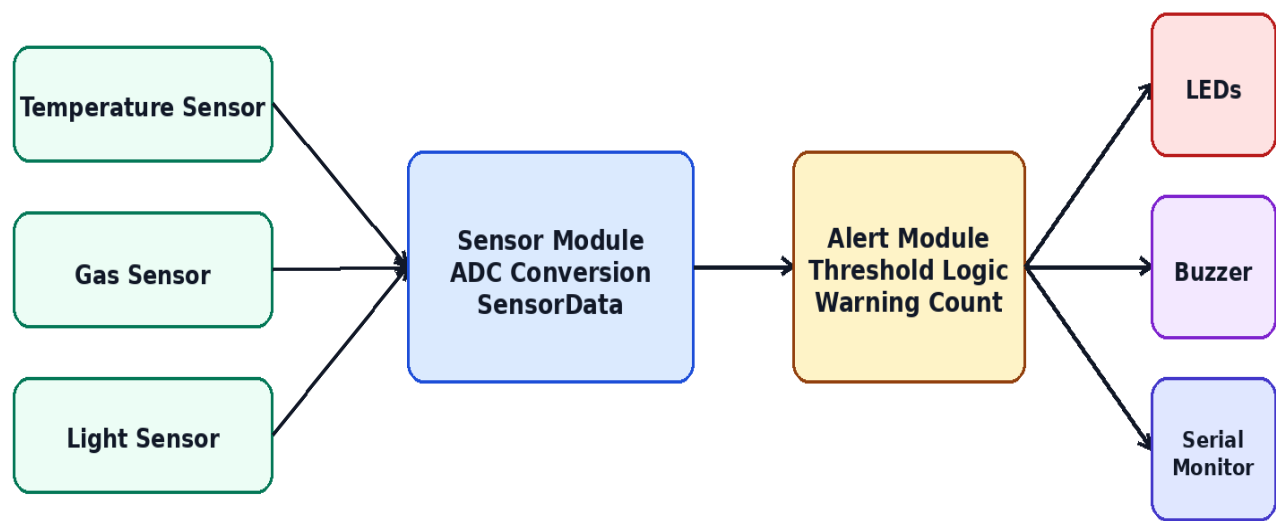


Figure 6: Multiple-file modular software architecture.

Modular Multi-Sensor System - Block Diagram



Main program coordinates modules while low-level functions remain separated.

Figure 5: Functional block diagram of the multi-sensor system.

Module/File	Responsibility
sketch.ino	Main initialization, periodic execution and system coordination
sensors.h / sensors.cpp	Sensor pin definitions, data structure and sensor-reading functions
alerts.h / alerts.cpp	Threshold evaluation, LED control, buzzer control and warning messages
diagram.json	ESP32 simulation circuit and component connections

Working Principle

Three potentiometers simulate temperature, gas and light sensors. The sensor module converts raw 12-bit ADC values into engineering percentages or temperature. The alert module compares these values with thresholds. A single abnormal condition generates a low-frequency alert, while multiple abnormal conditions generate a critical high-frequency warning.

Thresholds

Parameter	Normal Range	Warning Condition
Temperature	0-60 C	Above 60 C
Gas Level	0-60%	Above 60%
Light Level	25-100%	Below 25%

Algorithm

Multi-Sensor System Flowchart

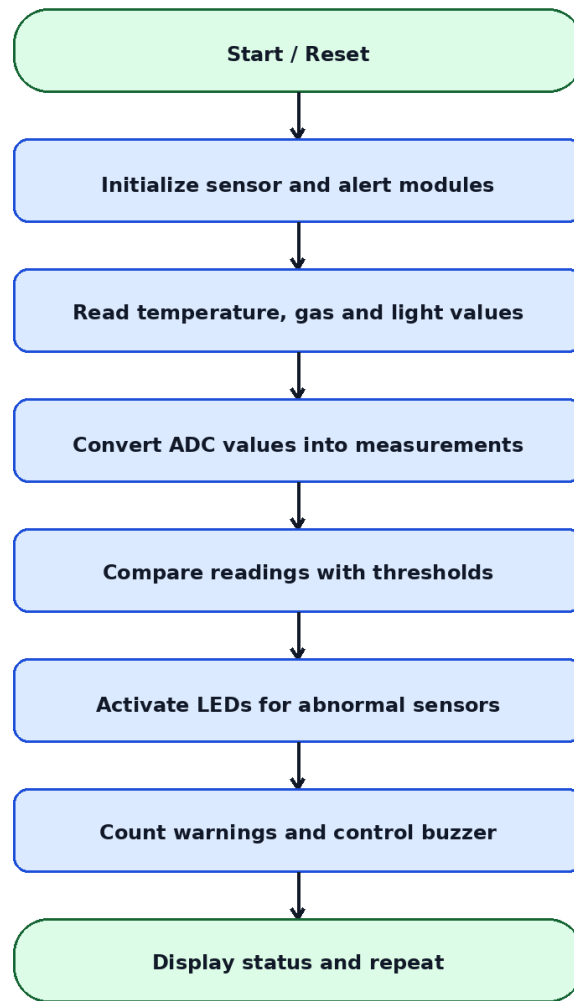


Figure 7: Flowchart of the modular multi-sensor process.

- Initialize Serial communication and all sensor/alert modules.
- Read temperature, gas and light sensor values.
- Convert raw ADC values into usable measurements.
- Display all values through the Serial Monitor.
- Compare each value with its configured threshold.
- Activate the corresponding warning LED.
- Count active warnings and operate the buzzer accordingly.
- Repeat the process every second.

Testing and Validation

Test Case	Sensor Condition	Expected Output
1	All values normal	All LEDs OFF and buzzer OFF
2	Temperature > 60 C	Temperature LED and one-condition alert
3	Gas > 60%	Gas LED and one-condition alert
4	Light < 25%	Light LED and one-condition alert
5	Two or more abnormal values	Multiple LEDs and critical buzzer

Scalability Analysis

- Separation of concerns keeps sensor logic independent from alert logic.
- A SensorData structure groups all measurements and creates a clear module interface.
- Adding a new sensor requires a new field, pin definition and read function without redesigning the alert hardware.
- Adding new output devices can be handled in the alert module without changing sensor acquisition.
- Multiple files enable parallel team development and simpler unit testing.
- Configuration constants can later be moved to a dedicated configuration file.
- For many sensors, arrays or classes can replace repeated code and reduce maintenance effort.
- For higher sampling rates, FreeRTOS tasks, queues and mutexes can separate acquisition, processing and communication.
- Memory usage grows linearly with the number of sensor records and remains predictable with fixed data structures.

Result

The multi-sensor system successfully displayed three sensor values and generated individual or critical alerts according to the configured thresholds.

Conclusion

The experiment demonstrates that modular Embedded C design improves code readability, organization, testing, and maintenance. Separate modules allow sensors and alert devices to be modified without affecting the complete system. New components can be added with only minor changes to the existing program. The modular structure also supports teamwork because different developers can work on separate files independently. It reduces code duplication and makes fault identification easier during debugging. Therefore, the system can be expanded from a simple three-sensor prototype into a larger and more reliable monitoring platform.

Program Code - Main Multi-Sensor Application

```
#define TEMP_PIN 34
#define GAS_PIN 35
#define LIGHT_PIN 32
#define TEMP_LED 25
#define GAS_LED 26
#define LIGHT_LED 27
#define BUZZER 14

struct SensorData {
  int temperature;
  int gas;
  int light;
};

SensorData readSensors() {
  SensorData d;
  d.temperature =
    map(analogRead(TEMP_PIN), 0, 4095, 0, 100);
  d.gas =
    map(analogRead(GAS_PIN), 0, 4095, 0, 100);
  d.light =
    map(analogRead(LIGHT_PIN), 0, 4095, 0, 100);
  return d;
}

void processAlerts(const SensorData &d) {
  bool t = d.temperature > 60;
  bool g = d.gas > 60;
  bool l = d.light < 25;

  digitalWrite(TEMP_LED, t);
  digitalWrite(GAS_LED, g);
  digitalWrite(LIGHT_LED, l);

  int warnings = t + g + l;
  if (warnings >= 2) tone(BUZZER, 1500);
  else if (warnings == 1) tone(BUZZER, 800);
  else noTone(BUZZER);

  Serial.print("Temperature: ");
  Serial.print(d.temperature);
  Serial.print(" C | Gas: ");
  Serial.print(d.gas);
  Serial.print("% | Light: ");
  Serial.print(d.light);
  Serial.println("%");

  if (warnings >= 2)
    Serial.println("CRITICAL: Multiple warnings");
  else if (warnings == 1)
    Serial.println("ALERT: One abnormal condition");
  else
    Serial.println("STATUS: All values normal");
}

void setup() {
  Serial.begin(115200);
  pinMode(TEMP_LED, OUTPUT);
  pinMode(GAS_LED, OUTPUT);
  pinMode(LIGHT_LED, OUTPUT);
  pinMode(BUZZER, OUTPUT);
  analogReadResolution(12);
  Serial.println("MODULAR MULTI-SENSOR SYSTEM");
}

void loop() {
  SensorData data = readSensors();
  processAlerts(data);
  delay(1000);
}
```

Suggested Multiple-File Organization

```
// sensors.h
struct SensorData {
  int temperature, gas, light;
};
void initializeSensors();
SensorData readAllSensors();

// sensors.cpp
#include "sensors.h"
void initializeSensors() {
  analogReadResolution(12);
}
SensorData readAllSensors() {
  return {
    map(analogRead(34), 0, 4095, 0, 100),
    map(analogRead(35), 0, 4095, 0, 100),
    map(analogRead(32), 0, 4095, 0, 100)
  };
}

// alerts.h
#include "sensors.h"
void initializeAlerts();
void processAlerts(const SensorData &data);

// sketch.ino
#include "sensors.h"
#include "alerts.h"
void setup() {
  Serial.begin(115200);
  initializeSensors();
  initializeAlerts();
}
void loop() {
  SensorData data = readAllSensors();
  processAlerts(data);
  delay(1000);
}
```

Simulation Output

The screenshot displays the Wokwi IDE interface for a project named 'New ESP32 Project - W'. The left pane shows the 'diagram.json' file with a JSON configuration for a modular multi-sensor system. The right pane shows a simulation of the circuit and a serial monitor window.

JSON Configuration (diagram.json):

```
5  "parts": [
6    {
7      "type": "resistor",
8      "id": "R1",
9      "value": "10k",
10     "label": "10K"
11   },
12   {
13     "type": "resistor",
14     "id": "R2",
15     "value": "10k",
16     "label": "10K"
17   },
18   {
19     "type": "resistor",
20     "id": "R3",
21     "value": "10k",
22     "label": "10K"
23   },
24   {
25     "type": "resistor",
26     "id": "R4",
27     "value": "10k",
28     "label": "10K"
29   },
30   {
31     "type": "resistor",
32     "id": "R5",
33     "value": "10k",
34     "label": "10K"
35   },
36   {
37     "type": "resistor",
38     "id": "R6",
39     "value": "10k",
40     "label": "10K"
41   },
42   {
43     "type": "resistor",
44     "id": "R7",
45     "value": "10k",
46     "label": "10K"
47   },
48   {
49     "type": "resistor",
50     "id": "R8",
51     "value": "10k",
52     "label": "10K"
53   },
54   {
55     "type": "resistor",
56     "id": "R9",
57     "value": "10k",
58     "label": "10K"
59   },
60   {
61     "type": "resistor",
62     "id": "R10",
63     "value": "10k",
64     "label": "10K"
65   },
66   {
67     "type": "resistor",
68     "id": "R11",
69     "value": "10k",
70     "label": "10K"
71   },
72   {
73     "type": "resistor",
74     "id": "R12",
75     "value": "10k",
76     "label": "10K"
77   },
78   {
79     "type": "resistor",
80     "id": "R13",
81     "value": "10k",
82     "label": "10K"
83   },
84   {
85     "type": "resistor",
86     "id": "R14",
87     "value": "10k",
88     "label": "10K"
89   },
90   {
91     "type": "resistor",
92     "id": "R15",
93     "value": "10k",
94     "label": "10K"
95   },
96   {
97     "type": "resistor",
98     "id": "R16",
99     "value": "10k",
100    "label": "10K"
101   },
102   {
103     "type": "resistor",
104     "id": "R17",
105     "value": "10k",
106     "label": "10K"
107   },
108   {
109     "type": "resistor",
110     "id": "R18",
111     "value": "10k",
112     "label": "10K"
113   },
114   {
115     "type": "resistor",
116     "id": "R19",
117     "value": "10k",
118     "label": "10K"
119   },
120   {
121     "type": "resistor",
122     "id": "R20",
123     "value": "10k",
124     "label": "10K"
125   },
126   {
127     "type": "resistor",
128     "id": "R21",
129     "value": "10k",
130     "label": "10K"
131   },
132   {
133     "type": "resistor",
134     "id": "R22",
135     "value": "10k",
136     "label": "10K"
137   },
138   {
139     "type": "resistor",
140     "id": "R23",
141     "value": "10k",
142     "label": "10K"
143   },
144   {
145     "type": "resistor",
146     "id": "R24",
147     "value": "10k",
148     "label": "10K"
149   },
150   {
151     "type": "resistor",
152     "id": "R25",
153     "value": "10k",
154     "label": "10K"
155   },
156   {
157     "type": "resistor",
158     "id": "R26",
159     "value": "10k",
160     "label": "10K"
161   },
162   {
163     "type": "resistor",
164     "id": "R27",
165     "value": "10k",
166     "label": "10K"
167   },
168   {
169     "type": "resistor",
170     "id": "R28",
171     "value": "10k",
172     "label": "10K"
173   },
174   {
175     "type": "resistor",
176     "id": "R29",
177     "value": "10k",
178     "label": "10K"
179   },
180   {
181     "type": "resistor",
182     "id": "R30",
183     "value": "10k",
184     "label": "10K"
185   },
186   {
187     "type": "resistor",
188     "id": "R31",
189     "value": "10k",
190     "label": "10K"
191   },
192   {
193     "type": "resistor",
194     "id": "R32",
195     "value": "10k",
196     "label": "10K"
197   },
198   {
199     "type": "resistor",
200     "id": "R33",
201     "value": "10k",
202     "label": "10K"
203   },
204   {
205     "type": "resistor",
206     "id": "R34",
207     "value": "10k",
208     "label": "10K"
209   },
210   {
211     "type": "resistor",
212     "id": "R35",
213     "value": "10k",
214     "label": "10K"
215   },
216   {
217     "type": "resistor",
218     "id": "R36",
219     "value": "10k",
220     "label": "10K"
221   },
222   {
223     "type": "resistor",
224     "id": "R37",
225     "value": "10k",
226     "label": "10K"
227   },
228   {
229     "type": "resistor",
230     "id": "R38",
231     "value": "10k",
232     "label": "10K"
233   },
234   {
235     "type": "resistor",
236     "id": "R39",
237     "value": "10k",
238     "label": "10K"
239   },
240   {
241     "type": "resistor",
242     "id": "R40",
243     "value": "10k",
244     "label": "10K"
245   },
246   {
247     "type": "resistor",
248     "id": "R41",
249     "value": "10k",
250     "label": "10K"
251   },
252   {
253     "type": "resistor",
254     "id": "R42",
255     "value": "10k",
256     "label": "10K"
257   },
258   {
259     "type": "resistor",
260     "id": "R43",
261     "value": "10k",
262     "label": "10K"
263   },
264   {
265     "type": "resistor",
266     "id": "R44",
267     "value": "10k",
268     "label": "10K"
269   },
270   {
271     "type": "resistor",
272     "id": "R45",
273     "value": "10k",
274     "label": "10K"
275   },
276   {
277     "type": "resistor",
278     "id": "R46",
279     "value": "10k",
280     "label": "10K"
281   },
282   {
283     "type": "resistor",
284     "id": "R47",
285     "value": "10k",
286     "label": "10K"
287   },
288   {
289     "type": "resistor",
290     "id": "R48",
291     "value": "10k",
292     "label": "10K"
293   },
294   {
295     "type": "resistor",
296     "id": "R49",
297     "value": "10k",
298     "label": "10K"
299   },
300   {
301     "type": "resistor",
302     "id": "R50",
303     "value": "10k",
304     "label": "10K"
305   },
306   {
307     "type": "resistor",
308     "id": "R51",
309     "value": "10k",
310     "label": "10K"
311   },
312   {
313     "type": "resistor",
314     "id": "R52",
315     "value": "10k",
316     "label": "10K"
317   },
318   {
319     "type": "resistor",
320     "id": "R53",
321     "value": "10k",
322     "label": "10K"
323   },
324   {
325     "type": "resistor",
326     "id": "R54",
327     "value": "10k",
328     "label": "10K"
329   },
330   {
331     "type": "resistor",
332     "id": "R55",
333     "value": "10k",
334     "label": "10K"
335   },
336   {
337     "type": "resistor",
338     "id": "R56",
339     "value": "10k",
340     "label": "10K"
341   },
342   {
343     "type": "resistor",
344     "id": "R57",
345     "value": "10k",
346     "label": "10K"
347   },
348   {
349     "type": "resistor",
350     "id": "R58",
351     "value": "10k",
352     "label": "10K"
353   },
354   {
355     "type": "resistor",
356     "id": "R59",
357     "value": "10k",
358     "label": "10K"
359   },
360   {
361     "type": "resistor",
362     "id": "R60",
363     "value": "10k",
364     "label": "10K"
365   },
366   {
367     "type": "resistor",
368     "id": "R61",
369     "value": "10k",
370     "label": "10K"
371   },
372   {
373     "type": "resistor",
374     "id": "R62",
375     "value": "10k",
376     "label": "10K"
377   },
378   {
379     "type": "resistor",
380     "id": "R63",
381     "value": "10k",
382     "label": "10K"
383   },
384   {
385     "type": "resistor",
386     "id": "R64",
387     "value": "10k",
388     "label": "10K"
389   },
390   {
391     "type": "resistor",
392     "id": "R65",
393     "value": "10k",
394     "label": "10K"
395   },
396   {
397     "type": "resistor",
398     "id": "R66",
399     "value": "10k",
400     "label": "10K"
401   },
402   {
403     "type": "resistor",
404     "id": "R67",
405     "value": "10k",
406     "label": "10K"
407   },
408   {
409     "type": "resistor",
410     "id": "R68",
411     "value": "10k",
412     "label": "10K"
413   },
414   {
415     "type": "resistor",
416     "id": "R69",
417     "value": "10k",
418     "label": "10K"
419   },
420   {
421     "type": "resistor",
422     "id": "R70",
423     "value": "10k",
424     "label": "10K"
425   },
426   {
427     "type": "resistor",
428     "id": "R71",
429     "value": "10k",
430     "label": "10K"
431   },
432   {
433     "type": "resistor",
434     "id": "R72",
435     "value": "10k",
436     "label": "10K"
437   },
438   {
439     "type": "resistor",
440     "id": "R73",
441     "value": "10k",
442     "label": "10K"
443   },
444   {
445     "type": "resistor",
446     "id": "R74",
447     "value": "10k",
448     "label": "10K"
449   },
450   {
451     "type": "resistor",
452     "id": "R75",
453     "value": "10k",
454     "label": "10K"
455   },
456   {
457     "type": "resistor",
458     "id": "R76",
459     "value": "10k",
460     "label": "10K"
461   },
462   {
463     "type": "resistor",
464     "id": "R77",
465     "value": "10k",
466     "label": "10K"
467   },
468   {
469     "type": "resistor",
470     "id": "R78",
471     "value": "10k",
472     "label": "10K"
473   },
474   {
475     "type": "resistor",
476     "id": "R79",
477     "value": "10k",
478     "label": "10K"
479   },
480   {
481     "type": "resistor",
482     "id": "R80",
483     "value": "10k",
484     "label": "10K"
485   },
486   {
487     "type": "resistor",
488     "id": "R81",
489     "value": "10k",
490     "label": "10K"
491   },
492   {
493     "type": "resistor",
494     "id": "R82",
495     "value": "10k",
496     "label": "10K"
497   },
498   {
499     "type": "resistor",
500     "id": "R83",
501     "value": "10k",
502     "label": "10K"
503   },
504   {
505     "type": "resistor",
506     "id": "R84",
507     "value": "10k",
508     "label": "10K"
509   },
510   {
511     "type": "resistor",
512     "id": "R85",
513     "value": "10k",
514     "label": "10K"
515   },
516   {
517     "type": "resistor",
518     "id": "R86",
519     "value": "10k",
520     "label": "10K"
521   },
522   {
523     "type": "resistor",
524     "id": "R87",
525     "value": "10k",
526     "label": "10K"
527   },
528   {
529     "type": "resistor",
530     "id": "R88",
531     "value": "10k",
532     "label": "10K"
533   },
534   {
535     "type": "resistor",
536     "id": "R89",
537     "value": "10k",
538     "label": "10K"
539   },
540   {
541     "type": "resistor",
542     "id": "R90",
543     "value": "10k",
544     "label": "10K"
545   },
546   {
547     "type": "resistor",
548     "id": "R91",
549     "value": "10k",
550     "label": "10K"
551   },
552   {
553     "type": "resistor",
554     "id": "R92",
555     "value": "10k",
556     "label": "10K"
557   },
558   {
559     "type": "resistor",
560     "id": "R93",
561     "value": "10k",
562     "label": "10K"
563   },
564   {
565     "type": "resistor",
566     "id": "R94",
567     "value": "10k",
568     "label": "10K"
569   },
570   {
571     "type": "resistor",
572     "id": "R95",
573     "value": "10k",
574     "label": "10K"
575   },
576   {
577     "type": "resistor",
578     "id": "R96",
579     "value": "10k",
580     "label": "10K"
581   },
582   {
583     "type": "resistor",
584     "id": "R97",
585     "value": "10k",
586     "label": "10K"
587   },
588   {
589     "type": "resistor",
590     "id": "R98",
591     "value": "10k",
592     "label": "10K"
593   },
594   {
595     "type": "resistor",
596     "id": "R99",
597     "value": "10k",
598     "label": "10K"
599   },
600   {
601     "type": "resistor",
602     "id": "R100",
603     "value": "10k",
604     "label": "10K"
605   },
606   {
607     "type": "resistor",
608     "id": "R101",
609     "value": "10k",
610     "label": "10K"
611   },
612   {
613     "type": "resistor",
614     "id": "R102",
615     "value": "10k",
616     "label": "10K"
617   },
618   {
619     "type": "resistor",
620     "id": "R103",
621     "value": "10k",
622     "label": "10K"
623   },
624   {
625     "type": "resistor",
626     "id": "R104",
627     "value": "10k",
628     "label": "10K"
629   },
630   {
631     "type": "resistor",
632     "id": "R105",
633     "value": "10k",
634     "label": "10K"
635   },
636   {
637     "type": "resistor",
638     "id": "R106",
639     "value": "10k",
640     "label": "10K"
641   },
642   {
643     "type": "resistor",
644     "id": "R107",
645     "value": "10k",
646     "label": "10K"
647   },
648   {
649     "type": "resistor",
650     "id": "R108",
651     "value": "10k",
652     "label": "10K"
653   },
654   {
655     "type": "resistor",
656     "id": "R109",
657     "value": "10k",
658     "label": "10K"
659   },
660   {
661     "type": "resistor",
662     "id": "R110",
663     "value": "10k",
664     "label": "10K"
665   },
666   {
667     "type": "resistor",
668     "id": "R111",
669     "value": "10k",
670     "label": "10K"
671   },
672   {
673     "type": "resistor",
674     "id": "R112",
675     "value": "10k",
676     "label": "10K"
677   },
678   {
679     "type": "resistor",
680     "id": "R113",
681     "value": "10k",
682     "label": "10K"
683   },
684   {
685     "type": "resistor",
686     "id": "R114",
687     "value": "10k",
688     "label": "10K"
689   },
690   {
691     "type": "resistor",
692     "id": "R115",
693     "value": "10k",
694     "label": "10K"
695   },
696   {
697     "type": "resistor",
698     "id": "R116",
699     "value": "10k",
700     "label": "10K"
701   },
702   {
703     "type": "resistor",
704     "id": "R117",
705     "value": "10k",
706     "label": "10K"
707   },
708   {
709     "type": "resistor",
710     "id": "R118",
711     "value": "10k",
712     "label": "10K"
713   },
714   {
715     "type": "resistor",
716     "id": "R119",
717     "value": "10k",
718     "label": "10K"
719   },
720   {
721     "type": "resistor",
722     "id": "R120",
723     "value": "10k",
724     "label": "10K"
725   },
726   {
727     "type": "resistor",
728     "id": "R121",
729     "value": "10k",
730     "label": "10K"
731   },
732   {
733     "type": "resistor",
734     "id": "R122",
735     "value": "10k",
736     "label": "10K"
737   },
738   {
739     "type": "resistor",
740     "id": "R123",
741     "value": "10k",
742     "label": "10K"
743   },
744   {
745     "type": "resistor",
746     "id": "R124",
747     "value": "10k",
748     "label": "10K"
749   },
750   {
751     "type": "resistor",
752     "id": "R125",
753     "value": "10k",
754     "label": "10K"
755   },
756   {
757     "type": "resistor",
758     "id": "R126",
759     "value": "10k",
760     "label": "10K"
761   },
762   {
763     "type": "resistor",
764     "id": "R127",
765     "value": "10k",
766     "label": "10K"
767   },
768   {
769     "type": "resistor",
770     "id": "R128",
771     "value": "10k",
772     "label": "10K"
773   },
774   {
775     "type": "resistor",
776     "id": "R129",
777     "value": "10k",
778     "label": "10K"
779   },
780   {
781     "type": "resistor",
782     "id": "R130",
783     "value": "10k",
784     "label": "10K"
785   },
786   {
787     "type": "resistor",
788     "id": "R131",
789     "value": "10k",
790     "label": "10K"
791   },
792   {
793     "type": "resistor",
794     "id": "R132",
795     "value": "10k",
796     "label": "10K"
797   },
798   {
799     "type": "resistor",
800     "id": "R133",
801     "value": "10k",
802     "label": "10K"
803   },
804   {
805     "type": "resistor",
806     "id": "R134",
807     "value": "10k",
808     "label": "10K"
809   },
810   {
811     "type": "resistor",
812     "id": "R135",
813     "value": "10k",
814     "label": "10K"
815   },
816   {
817     "type": "resistor",
818     "id": "R136",
819     "value": "10k",
820     "label": "10K"
821   },
822   {
823     "type": "resistor",
824     "id": "R137",
825     "value": "10k",
826     "label": "10K"
827   },
828   {
829     "type": "resistor",
830     "id": "R138",
831     "value": "10k",
832     "label": "10K"
833   },
834   {
835     "type": "resistor",
836     "id": "R139",
837     "value": "10k",
838     "label": "10K"
839   },
840   {
841     "type": "resistor",
842     "id": "R140",
843     "value": "10k",
844     "label": "10K"
845   },
846   {
847     "type": "resistor",
848     "id": "R141",
849     "value": "10k",
850     "label": "10K"
851   },
852   {
853     "type": "resistor",
854     "id": "R142",
855     "value": "10k",
856     "label": "10K"
857   },
858   {
859     "type": "resistor",
860     "id": "R143",
861     "value": "10k",
862     "label": "10K"
863   },
864   {
865     "type": "resistor",
866     "id": "R144",
867     "value": "10k",
868     "label": "10K"
869   },
870   {
871     "type": "resistor",
872     "id": "R145",
873     "value": "10k",
874     "label": "10K"
875   },
876   {
877     "type": "resistor",
878     "id": "R146",
879     "value": "10k",
880     "label": "10K"
881   },
882   {
883     "type": "resistor",
884     "id": "R147",
885     "value": "10k",
886     "label": "10K"
887   },
888   {
889     "type": "resistor",
890     "id": "R148",
891     "value": "10k",
892     "label": "10K"
893   },
894   {
895     "type": "resistor",
896     "id": "R149",
897     "value": "10k",
898     "label": "10K"
899   },
900   {
901     "type": "resistor",
902     "id": "R150",
903     "value": "10k",
904     "label": "10K"
905   },
906   {
907     "type": "resistor",
908     "id": "R151",
909     "value": "10k",
910     "label": "10K"
911   },
912   {
913     "type": "resistor",
914     "id": "R152",
915     "value": "10k",
916     "label": "10K"
917   },
918   {
919     "type": "resistor",
920     "id": "R153",
921     "value": "10k",
922     "label": "10K"
923   },
924   {
925     "type": "resistor",
926     "id": "R154",
927     "value": "10k",
928     "label": "10K"
929   },
930   {
931     "type": "resistor",
932     "id": "R155",
933     "value": "10k",
934     "label": "10K"
935   },
936   {
937     "type": "resistor",
938     "id": "R156",
939     "value": "10k",
940     "label": "10K"
941   },
942   {
943     "type": "resistor",
944     "id": "R157",
945     "value": "10k",
946     "label": "10K"
947   },
948   {
949     "type": "resistor",
950     "id": "R158",
951     "value": "10k",
952     "label": "10K"
953   },
954   {
955     "type": "resistor",
956     "id": "R159",
957     "value": "10k",
958     "label": "10K"
959   },
960   {
961     "type": "resistor",
962     "id": "R160",
963     "value": "10k",
964     "label": "10K"
965   },
966   {
967     "type": "resistor",
968     "id": "R161",
969     "value": "10k",
970     "label": "10K"
971   },
972   {
973     "type": "resistor",
974     "id": "R162",
975     "value": "10k",
976     "label": "10K"
977   },
978   {
979     "type": "resistor",
980     "id": "R163",
981     "value": "10k",
982     "label": "10K"
983   },
984   {
985     "type": "resistor",
986     "id": "R164",
987     "value": "10k",
988     "label": "10K"
989   },
990   {
991     "type": "resistor",
992     "id": "R165",
993     "value": "10k",
994     "label": "10K"
995   },
996   {
997     "type": "resistor",
1000    "id": "R166",
1001    "value": "10k",
1002    "label": "10K"
1003   },
1004   {
1005     "type": "resistor",
1006     "id": "R167",
1007     "value": "10k",
1008     "label": "10K"
1009   },
1010   {
1011     "type": "resistor",
1012     "id": "R168",
1013     "value": "10k",
1014     "label": "10K"
1015   },
1016   {
1017     "type": "resistor",
1018     "id": "R169",
1019     "value": "10k",
1020     "label": "10K"
1021   },
1022   {
1023     "type": "resistor",
1024     "id": "R170",
1025     "value": "10k",
1026     "label": "10K"
1027   },
1028   {
1029     "type": "resistor",
1030     "id": "R171",
1031     "value": "10k",
1032     "label": "10K"
1033   },
1034   {
1035     "type": "resistor",
1036     "id": "R172",
1037     "value": "10k",
1038     "label": "10K"
1039   },
1040   {
1041     "type": "resistor",
1042     "id": "R173",
1043     "value": "10k",
1044     "label": "10K"
1045   },
1046   {
1047     "type": "resistor",
1048     "id": "R174",
1049     "value": "10k",
1050     "label": "10K"
1051   },
1052   {
1053     "type": "resistor",
1054     "id": "R175",
1055     "value": "10k",
1056     "label": "10K"
1057   },
1058   {
1059     "type": "resistor",
1060     "id": "R176",
1061     "value": "10k",
1062     "label": "10K"
1063   },
1064   {
1065     "type": "resistor",
1066     "id": "R177",
1067     "value": "10k",
1068     "label": "10K"
1069   },
1070   {
1071     "type": "resistor",
1072     "id": "R178",
1073     "value": "10k",
1074     "label": "10K"
1075   },
1076   {
1077     "type": "resistor",
1078     "id": "R179",
1079     "value": "10k",
1080     "label": "10K"
1081   },
1082   {
1083     "type": "resistor",
1084     "id": "R180",
1085     "value": "10k",
1086     "label": "10K"
1087   },
1088   {
1089     "type": "resistor",
1090     "id": "R181",
1091     "value": "10k",
1092     "label": "10K"
1093   },
1094   {
1095     "type": "resistor",
1096     "id": "R182",
1097     "value": "10k",
1098     "label": "10K"
1099   },
1100   {
1101     "type": "resistor",
1102     "id": "R183",
1103     "value": "10k",
1104     "label": "10K"
1105   },
1106   {
1107     "type": "resistor",
1108     "id": "R184",
1109     "value": "10k",
1110     "label": "10K"
1111   },
1112   {
1113     "type": "resistor",
1114     "id": "R185",
1115     "value": "10k",
1116     "label": "10K"
1117   },
1118   {
1119     "type": "resistor",
1120     "id": "R186",
1121     "value": "10k",
1122     "label": "10K"
1123   },
1124   {
1125     "type": "resistor",
1126     "id": "R187",
1127     "value": "10k",
1128     "label": "10K"
1129   },
1130   {
1131     "type": "resistor",
1132     "id": "R188",
1133     "value": "10k",
1134     "label": "10K"
1135   },
1136   {
1137     "type": "resistor",
1138     "id": "R189",
1139     "value": "10k",
1140     "label": "10K"
1141   },
1142   {
1143     "type": "resistor",
1144     "id": "R190",
1145     "value": "10k",
1146     "label": "10K"
1147   },
1148   {
1149     "type": "resistor",
1150     "id": "R191",
1151     "value": "10k",
1152     "label": "10K"
1153   },
1154   {
1155     "type": "resistor",
1156     "id": "R192",

```